

TileLang典型算子性能优化

tile-ai社区Collaborator 张雨晨

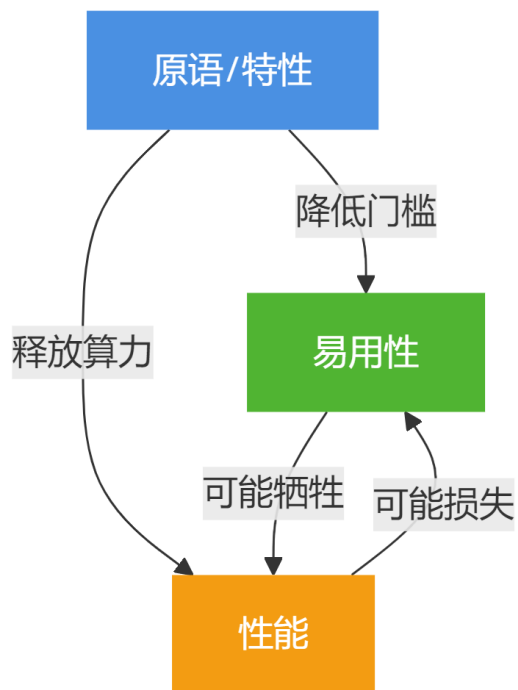
2026.4.21

1 背景

2 性能优化方法

3 性能优化案例

背景



TileLang-Ascend

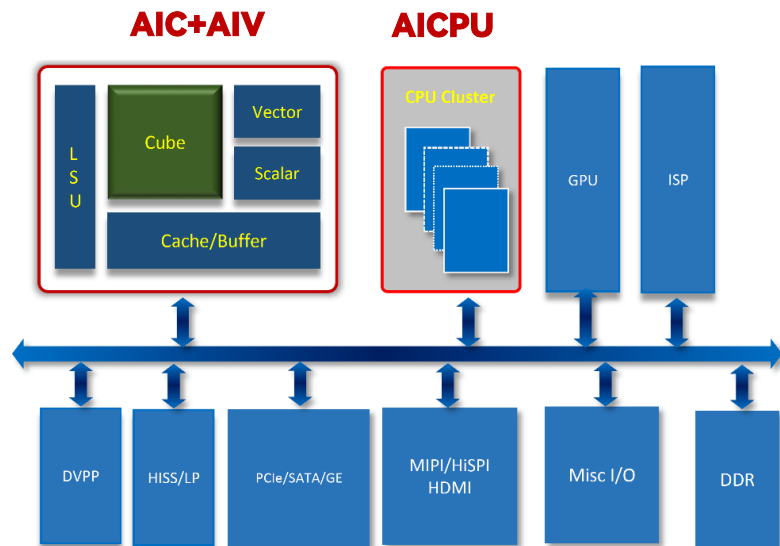
- TileLang-Ascend 基于 TileLang 的 Python 式语法与 TVM 编译器基础架构，帮助开发者高效构建专为 Ascend 处理器定制的高性能 AI 计算内核。
- 让开发者能够专注于提高开发效率，同时又不牺牲 NPU 上实现较高性能所需的底层优化；
- 提供T.Pipelined原语、自动插入核间核内同步等特性降低性能优化门槛，助力在expert+developer混合编程模式下达到较好性能。

1 背景

2 性能优化方法

3 性能优化案例

性能优化方法

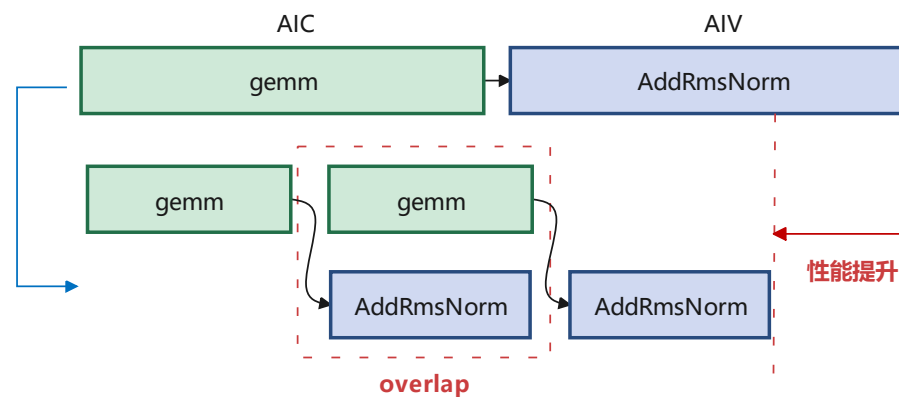


CV融合类算子(如FA、SFA):

- 核间流水优化
- 核内流水优化

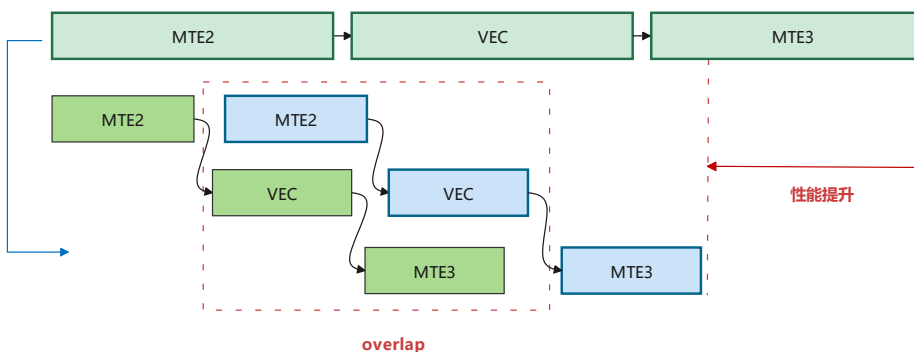
纯Cube/Vector算子(如gemm、Add):

- 核内流水优化



Inter-core

- 核间pipeline, **Cube**和**Vector**间任务overlap, 调节并行度和核间同步次数



Intra-core

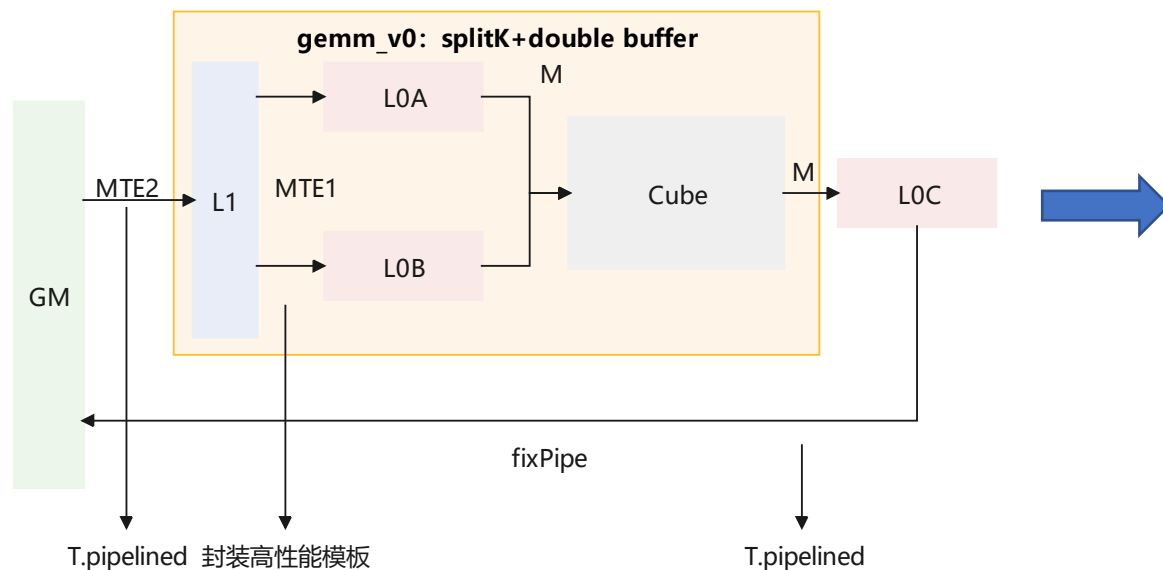
- **Cube/Vector**: 核内流水掩盖
- **Cube**: L1内存常驻, splitK, 调整核间及核内切分策略
- **Vector**: 算法优化, 指令向量化, 离散访存连续化

Fixed core

- with `T.Kernel(CORE_NUM, is_npu=True)` as (cid, vid): 减少初始化开销、显存可控、数据常驻 L2

性能优化方法 —— Cube核内性能优化

Cube矩阵运算(Q @ K): 数据切分与流水设计



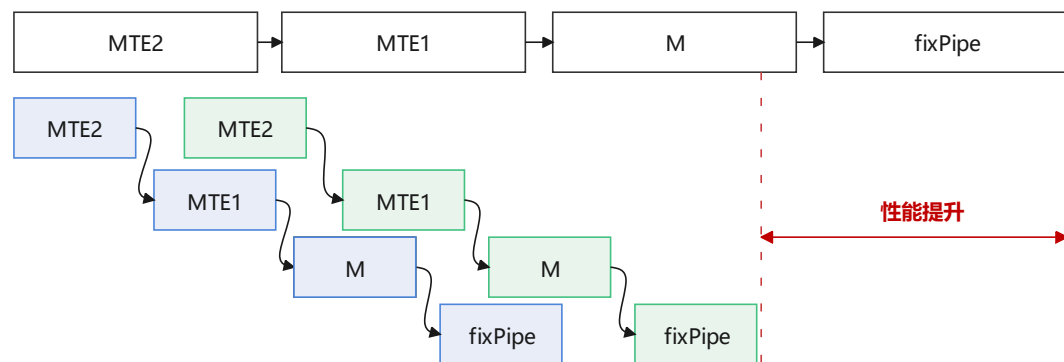
- **GM → L1 / LOC → GM: 多核间数据切分**
- **L1 → L0: 单核内多次计算, 核内数据切分**
- **GM → L1 / L1 → L0 / L0 → GM: 均可开启多buffer流水**

<https://gitcode.com/cann>

gemm_v0为封装好的固定模板: 追求更好的性能可以采用混合编程模式

- **核间数据切分: 找到合适的block_M, block_N, block_K, 尽量让L1全载, 提高搬运效率**
- **核内数据切分: L1大小大于L0, L1全载时数据需要分多次搬入L0**

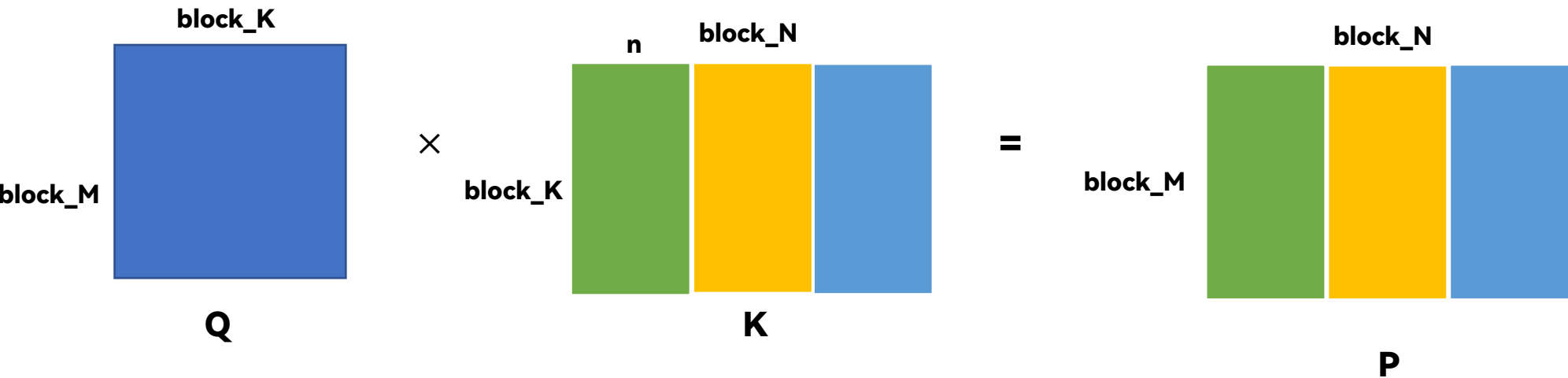
- **开启多级流水实现overlap**



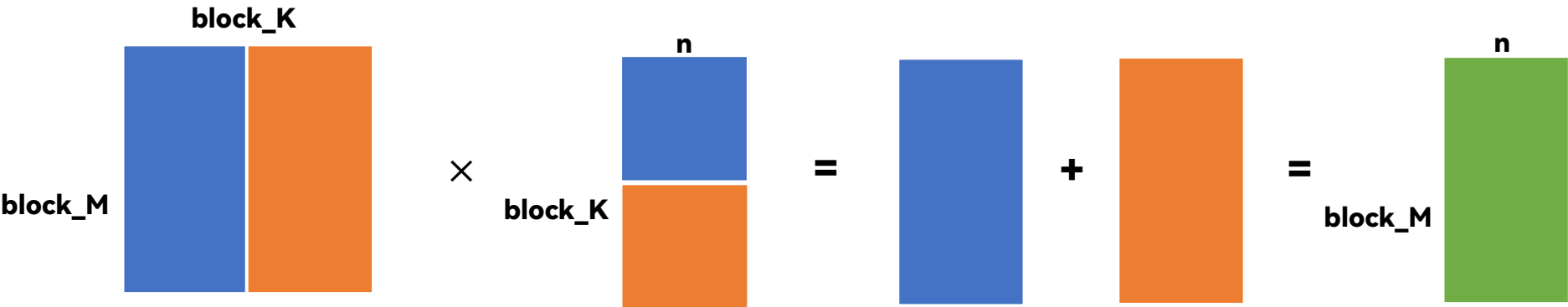
性能优化方法 —— Cube核内性能优化

Cube矩阵运算(Q @ K): 切分策略

- L1内存常驻: gemm运算分多块, 块间Q矩阵常驻, 减少访存



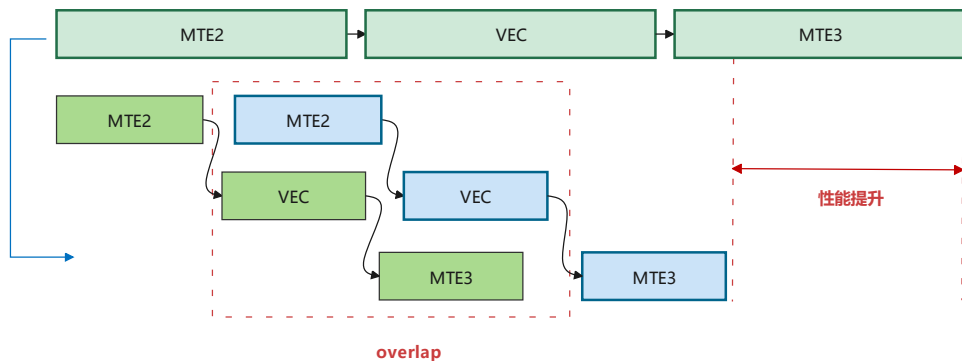
- splitK: 单次个gemm运算内部切分K, 实现核内数据块间overlap



性能优化方法 —— Vector核内性能优化

Vector运算

- 核内多buffer: MTE2、VEC、MTE3间overlap



for i in T.Pipelined(loop_num, num_stages):

copy_gm_to_ub

compute

num_stages < loop_num

copy_ub_to_gm

计算与搬运时间不均匀: num_stages越大, 气泡越小,
但会占用更多内存

尝试不同的num_stages寻找最优解

- 算法优化: 减少指令数量/使用更高效的替代指令

Axpy(y, x, a): $y = ax + y$

优化前: 两条指令

```
T.tile.mul(acc_s_ub, acc_s_ub, sm_scale)
T.tile.sub(acc_s_ub, acc_s_ub, m_i_2d)
```

优化后: 一条指令

```
T.tile.axpy(acc_s_ub, m_i_2d, sm_scale)
```

- 指令向量化: 离散操作连续化, 高效的tile操作替换低效scalar操作

优化前, 对m_i进行多次getvalue的scalar操作

```
for h_i in range(block_M // 2):
    T.tile.sub(acc_s_ub[h_i, :], acc_s_ub[h_i, :], m_i[h_i])
```

优化后, 转化为一次tile操作

```
T.tile.broadcast(m_i_2d, m_i, tmp_ub)
T.tile.sub(acc_s_ub, acc_s_ub, m_i_2d)
```

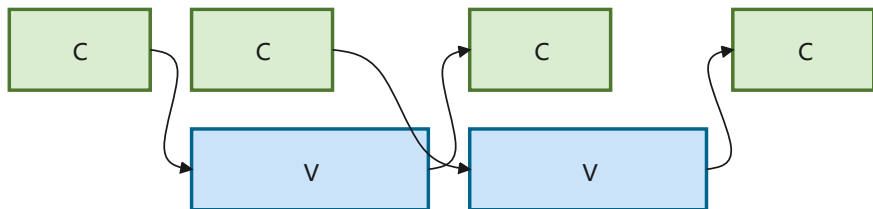
多次循环set_value → 单次tile操作

性能优化方法 —— 核间性能优化

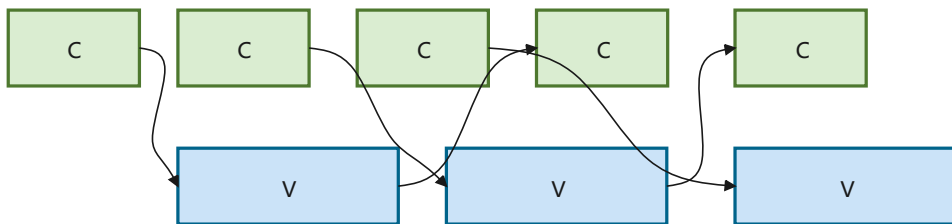
➤ 核间pipeline: *for i in T.Pipelined(loop_num, num_stages, **cross_interval**)*

CV执行时间不均衡时，适当增大num_stages可以减小气泡

num_stages=2: 由于vector耗时大于cube，C核存在气泡

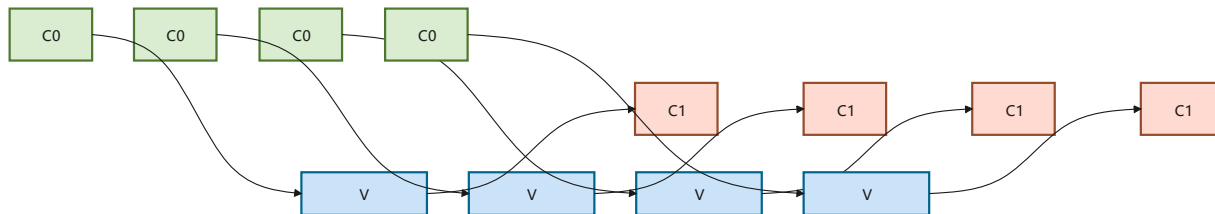


num_stages=3: 气泡减小，并行效果更好

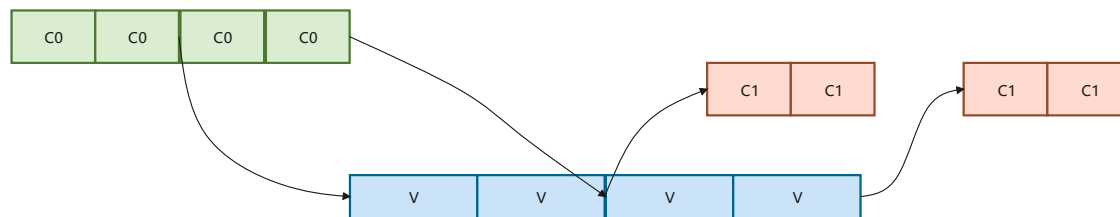


跨核同步可能带来较大的开销，设置cross_interval控制同步次数

cross_interval=1: 单次任务同步，核间同步存在调度开销，不是计算结束后马上就能wait到flag，造成气泡



cross_interval=2: 两次任务同步，减少核间同步次数带来的开销，但并行度变差



性能优化方法 —— Fixed Core

```
@T.prim_func
def tl_kernel(
    ...
    # Workspace 按逻辑任务数分配, block_num 可能远大于物理核数
    workspace: T.Tensor([block_num, block_M, block_N], dtype),
):
    # ↓ alloc_buffer / annotate_address 随 block_num 展开, 每个逻辑任务各执行一次
    buf = T.alloc_l1([block_M, block_N], dtype)
    T.annotate_layout({buf: ...})

    with T.Kernel(block_num, is_npu=True) as (cid, vid):
        # 每个逻辑任务拥有独立的 workspace 切片
        T.copy(result, workspace[cid, :, :])
```

➤ 默认模式下, 当逻辑任务数远大于物理核数时, 会产生性能问题:

- 初始化冗余: 随任务数重复执行
- 显存膨胀: Workspace 随任务数线性增长
- 访存瓶颈: 中间张量溢出 L2 到 片上内存中, Cube-Vector 交互延迟高

```
@T.prim_func
def tl_kernel(
    ...
    # Workspace 按物理核数分配, 固定 24 份, 尽可能常驻 L2
    workspace: T.Tensor([core_num, block_M, block_N], dtype),
):
    # ↓ alloc_buffer / annotate_address 仅在 24 个物理核上各执行一次
    buf = T.alloc_l1([block_M, block_N], dtype)
    T.annotate_layout({buf: ...})

    with T.Kernel(core_num, is_npu=True) as (cid, vid):
        # 手动将逻辑任务均摊到物理核, 每个核复用同一份 workspace
        single_core_load = T.ceildiv(block_num, core_num)
        for block_idx in T.serial(cid * single_core_load, (cid + 1) * single_core_load):
            ...
            T.copy(result, workspace[cid, :, :]) # workspace[cid] 被复用
```

➤ Fixed Core模式通过 T.Kernel(core_num, is_npu=True) 将计算任务约束在固定数量的物理核心上, 解决上述问题:

- 减少初始化: 仅物理核(如 24 核) 各执行一次
- 显存可控: Workspace 固定为 core_num 份
- 数据常驻 L2: 中间张量片上复用, 避免片上内存置换

性能优化方法

T.Pipelined: tileLang 提供的用于开启核内/核间流水的原语

定义

Interface

```
for var in T.Pipelined(loop_iterations: int, num_stages: int, cross_interval: int = 1):
```

Parameter	Type	Default	Description
loop_iterations	int	Required	Total number of loop iterations
num_stages	int	Required	Number of pipeline stages for double buffering
cross_interval	int	1	Interval for cross-core synchronization (only effective in inter-core pipeline)

https://github.com/tile-ai/tilelang-ascend/blob/ascendc_pto/docs/tutorials/t_pipelined.md

核内

Intra-core Pipeline

Intra-core pipeline overlaps memory access and computation within a single core. When `num_stages=2`, the execution pattern is:

```
for k in T.Pipelined(loop_k, num_stages=2):
    T.copy(A[bx * block_M, k * block_K], A_L1)
    T.copy(B[k * block_K, by * block_N], B_L1)

    T.barrier_all()
    if k == 0:
        T.gemm_v0(A_L1, B_L1, C_L0, init=True)
    else:
        T.gemm_v0(A_L1, B_L1, C_L0)

    T.barrier_all()
```

不支持嵌套使用

Nested Pattern (Not Supported)

In nested pattern, both inter-core and intra-core pipeline are enabled using `T.Pipelined`:

```
# ❌ Not supported: T.Pipelined nested, enabling both inter-core and intra-core
for k in T.Pipelined(num_iters, num_stages=4): # Inter-core pipeline
    # T.Pipelined enables inter-core sync

    # Nested T.Pipelined for intra-core double buffering
    for side in T.Pipelined(2, num_stages=2): # ❌ Not supported
        T.copy(K[side], k_l1)
        T.copy(k_l1, l0b[side])
        T.mma(l0a[side], l0b[side], l0c[side], init=(side == 0))
```

核间

Inter-core Pipeline

Inter-core pipeline overlaps computation between Cube and Vector cores through workspace buffers.

```
for k in T.Pipelined(T.ceildiv(seq_len, block_N), num_stages=2):
    # Cube: write to workspace
    T.copy(K[bz, by, k * block_N:(k + 1) * block_N, :], k_l1)
    T.gemm_v0(q_l1, k_l1, acc_s_l0c, transpose_B=True, init=True)
    T.copy(acc_s_l0c, workspace_1[cid, :, :])

    # Vector: read from workspace
    T.copy(workspace_1[cid, vid * block_M // 2:vid * block_M // 2 + block_M // 2, :], acc_s_ub_)
    T.tile.add(acc_s_ub, acc_s_ub, acc_s_ub_)
```

1 背景

2 性能优化方法

3 性能优化案例

FA算子性能优化案例

FlashAttention是一种IO感知的精确注意力计算算法，由斯坦福大学Tri Dao团队于2022年提出。其核心目标是：在不牺牲精度的前提下，大幅减少片上内存访存次数和内存占用。

注意力机制的计算公式为：

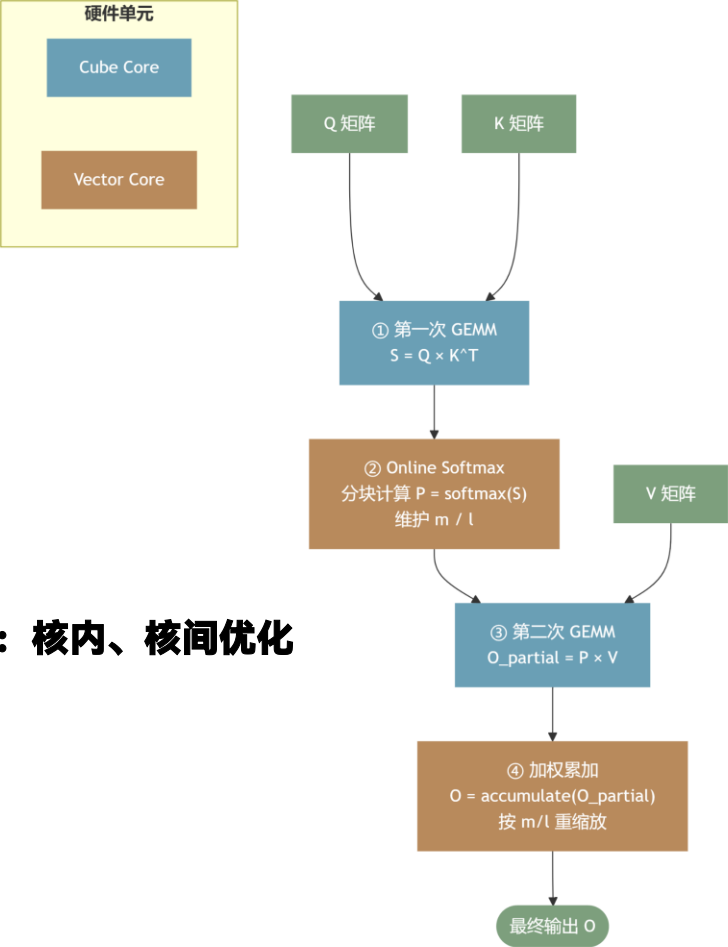
$$Attention(Q, K, V) = Softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

tilelang-ascend / examples / flash_attention / fa_opt /

pbbb205 [Feature]Remove tmp argument in vector core API (#676)

Name	Last commit message
..	
bench.sh	Add FA high performance (#698)
bench_mark.md	Add cross_interval for pipeline (#711)
bench_mark_zh.md	Add cross_interval for pipeline (#711)
flash_attention_performance_optimization.md	Update doc for high perf FA (#706)
flash_attention_performance_optimization_zh.md	Update doc for high perf FA (#706)
flash_attn_bhscd_ascendc.py	Add FA high performance (#698)
flash_attn_bhscd_auto_pipeline_h16_d128.py	[Feature]Remove tmp argument in vector core API (#676)
flash_attn_bhscd_auto_pipeline_h32_d512.py	[Feature]Remove tmp argument in vector core API (#676)
flash_attn_bhscd_expert_h16_d128.py	[Feature]Remove tmp argument in vector core API (#676)
plot.py	[Benchmark] Add FA benchmark and optimize FA in developer mode (#305)
run.py	Add FA high performance (#698)

CV融合算子：核内、核间优化



https://github.com/tile-ai/tilelang-ascend/tree/ascendc_pto/examples/flash_attention/fa_opt

FA算子性能优化案例

B*S=16K时开箱性能达Ascend C手写算子的97%:

shape	TileLang(us)	Ascend C(us)	性能现状
b: 2, q_h: 32, kv_h: 32, d: 512, s: 8192	100633	98020	97.40%
b: 4, q_h: 32, kv_h: 32, d: 512, s: 4096	51177	50054	97.81%
b: 8, q_h: 32, kv_h: 32, d: 512, s: 2048	26636	25671	96.38%

典型shape下开箱性能为Ascend C手写算子的36% :

shape	TileLang(us)	Ascend C(us)	性能现状
b: 1, q_h: 12, kv_h: 1, d: 128, s: 32768 (32K)	102682	37555	36.57%
b: 1, q_h: 12, kv_h: 1, d: 128, s: 65536 (64K)	407683	149578	36.69%
b: 1, q_h: 12, kv_h: 1, d: 128, s: 131072 (128K)	1644569	600018	36.48%

优化后典型shape下性能达Ascend C手写算子的80% :

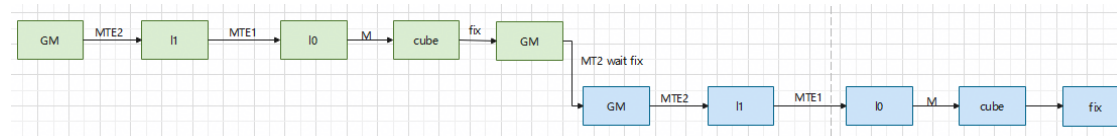
shape	TileLang(us)	Ascend C(us)	性能现状
b: 1, q_h: 12, kv_h: 1, d: 128, s: 32768 (32K)	46643	37555	80.52%
b: 1, q_h: 12, kv_h: 1, d: 128, s: 65536 (64K)	185188	149578	80.77%
b: 1, q_h: 12, kv_h: 1, d: 128, s: 131072 (128K)	741211	600018	80.95%

FA算子性能优化案例

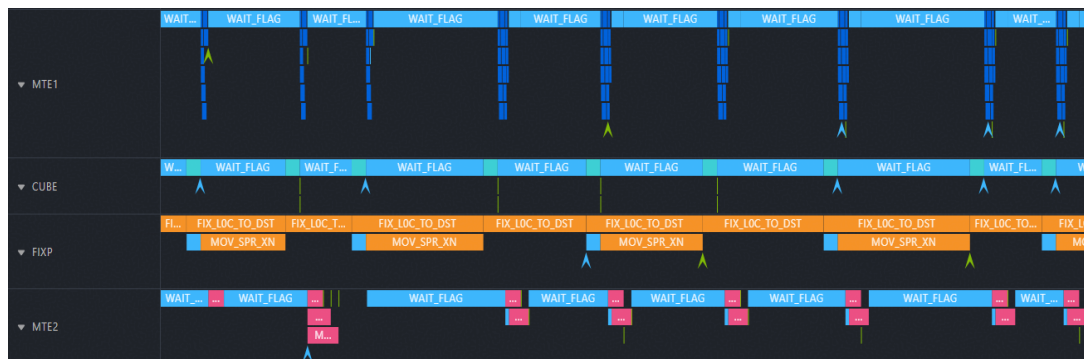
➤ 优化Cube核内流水，gemm细化同步



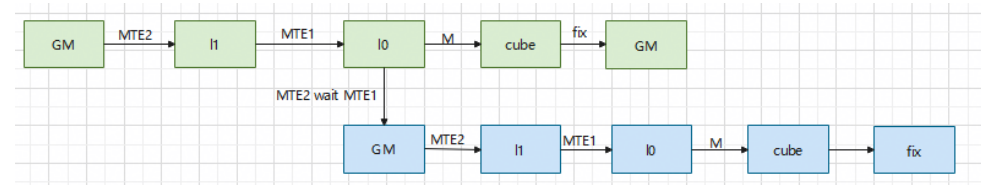
优化前MTE2、MTE1、M、fix流水之间完全串行，流水排布为：



轮次间完全串行，有多余的MTE2 wait fix操作，没有overlap



优化后开启L0层级的double buffer，实现轮次间的流水并行



FixPipe bound

性能提升：36% -> 47%

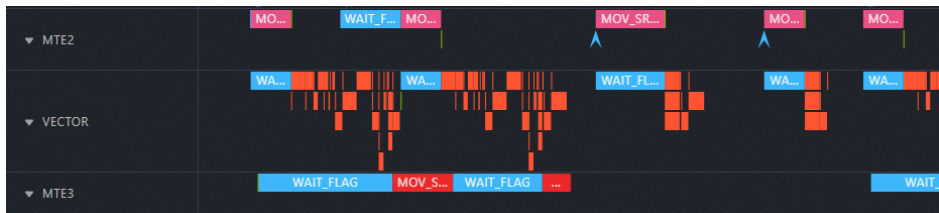
<https://gitcode.com/cann>

CANN

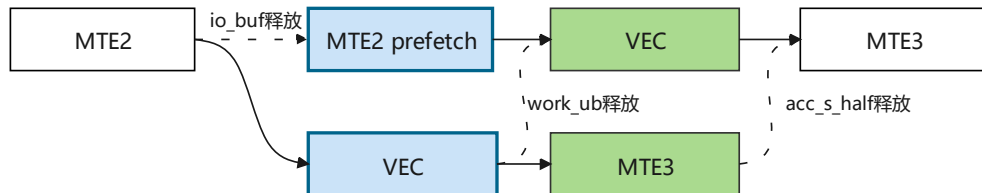
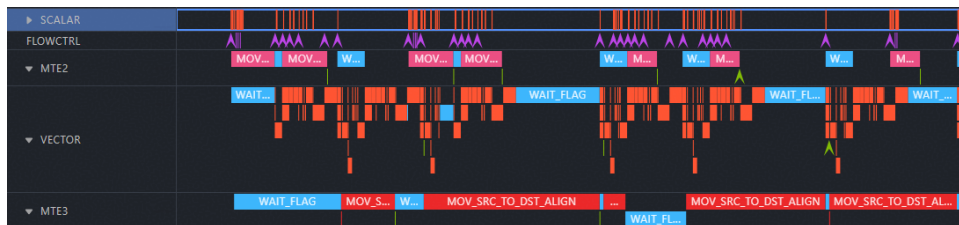
FA算子性能优化案例

➤ Vector开启核内流水(double buffer) :

优化前：串行流水



优化后：MTE2、vector、MTE3流水线并行



性能提升：47% -> 55%

<https://gitcode.com/cann>

```
T.copy(workspace_1[cid, vid * block_M // 2 : vid * block_M // 2 + block_M // 2, :], acc_s_ub_)
T.tile.add(acc_s_ub, acc_s_ub, acc_s_ub_)
T.tile.mul(acc_s_ub, acc_s_ub, sm_scale)
T.reduce_max(acc_s_ub, m_i, dim=-1)
T.tile.max(m_i, m_i, m_i_prev)
T.tile.sub(m_i_prev, m_i_prev, m_i)
T.tile.exp(m_i_prev, m_i_prev)

T.tile.broadcast(m_i_2d, m_i)
T.tile.sub(acc_s_ub, acc_s_ub, m_i_2d)

T.tile.exp(acc_s_ub, acc_s_ub)
T.reduce_sum(acc_s_ub, sumexp_i_ub, dim=-1)
T.tile.mul(sumexp, sumexp, m_i_prev)
T.tile.add(sumexp, sumexp, sumexp_i_ub)

T.copy(acc_s_ub, acc_s_half)
T.copy(acc_s_half, workspace_2[cid, vid * block_M // 2 : vid * block_M // 2 + block_M // 2, :])
```

MTE2

VEC

MTE3

```
# ---- V: softmax(ws1[0]) ----
T.copy(workspace_1[cid, 0, vid * half_M : vid * half_M + half_M, :], io_buf)
T.copy(io_buf, work_ub)
T.copy(workspace_1[cid, 1, vid * half_M : vid * half_M + half_M, :], io_buf)
T.reduce_max(work_ub, neg_sm[0, :, :], dim=-1)
T.tile.mul(neg_sm[0, :, :], neg_sm[0, :, :], -sm_scale)
T.tile.min(neg_sm[0, :, :], neg_sm[0, :, :], neg_sm[1, :, :])
T.tile.broadcast(buf_2d, neg_sm[0, :, :])
T.tile.axy(buf_2d, work_ub, sm_scale)
T.tile.exp(work_ub, buf_2d)
T.copy(work_ub, acc_s_half)
T.copy(acc_s_half, workspace_2[cid, 0, vid * half_M : vid * half_M + half_M, :])
T.reduce_sum(work_ub, sumexp_is[0, :, :], dim=-1)
T.tile.sub(r_factors[0, :, :], neg_sm[0, :, :], neg_sm[1, :, :])

# ---- V: softmax(ws1[1]) ----
T.copy(io_buf, work_ub)
T.reduce_max(work_ub, neg_sm[1, :, :], dim=-1)
T.tile.mul(neg_sm[1, :, :], neg_sm[1, :, :], -sm_scale)
T.tile.min(neg_sm[1, :, :], neg_sm[1, :, :], neg_sm[0, :, :])
T.tile.broadcast(buf_2d, neg_sm[1, :, :])
T.tile.axy(buf_2d, work_ub, sm_scale)
T.tile.exp(work_ub, buf_2d)
T.copy(work_ub, acc_s_half)
T.copy(acc_s_half, workspace_2[cid, 1, vid * half_M : vid * half_M + half_M, :])
T.reduce_sum(work_ub, sumexp_is[1, :, :], dim=-1)
T.tile.sub(r_factors[1, :, :], neg_sm[1, :, :], neg_sm[0, :, :])
```

MTE2

MTE2 prefetch

VEC

MTE3

VEC

MTE3

CANN

FA算子性能优化案例

➤ 优化Vector算法，减少指令数

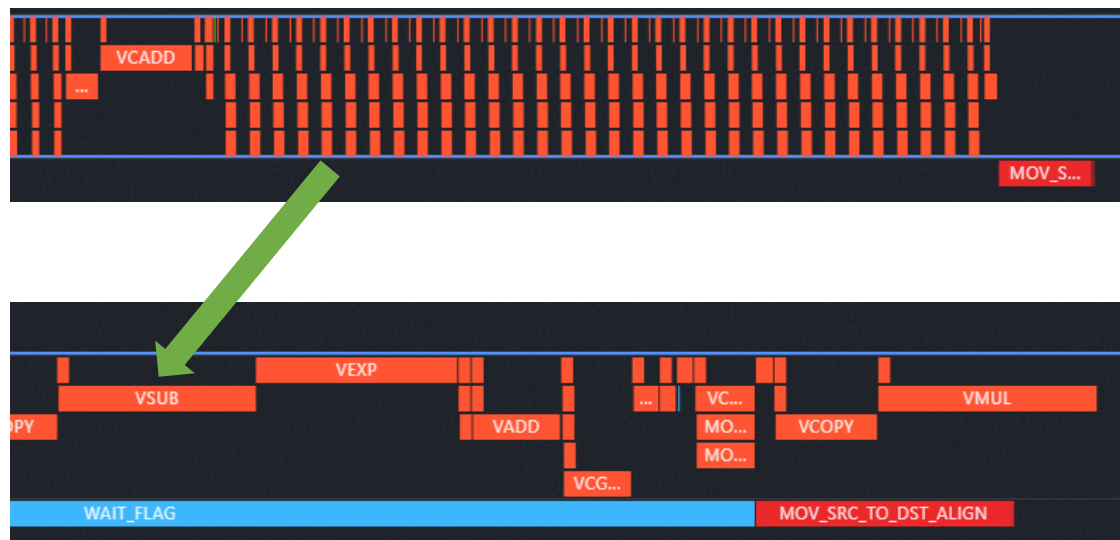
用一条axpy指令代替之前的多条计算指令

`axpy(y, x, a): y = a * x + y`

优化前: `tmp = m* sm_scale; result = tmp - max`

优化后: `result = axpy(-max, m, sm_scale)`

➤ 指令向量化



`getValue+subs` → `tils.sub`

多次scalar操作 → 一次tile操作

性能提升: 55% -> 63%

FA算子性能优化案例

➤ 核间pipeline优化



优化前: num_stages = 2, cross_interval = 1

- Cube和Vec之间存在气泡
- 注释掉所有计算/搬运, 只剩核间同步, 发现耗时占算子执行耗时的50%以上, 同步开销较大



优化后: num_stages = 8, cross_interval = 2

- Cube和Vector任务紧凑, Cube核几乎无气泡
- 同步耗时显著降低, 占算子执行耗时的20%左右

<https://gitcode.com/cann>

性能提升: 63% -> 80%

CANN

FA算子性能优化案例

通过系统性优化，FA 算子在保持 TileLang 的高开发效率前提下，达到了 Ascend C 原生算子**80%**的性能，混合编程模式下性能达到原生算子**60%**。

优化项	L1 内存常驻	指令向量化	多 Buffer	核内冗余同步消除	CV pipelined	优化核间同步下发次数	减少指令数	性能 (A3)
flash_attn_bhsd_expert_h16_d128.py	√	√	√	√	√	√	√	80%
flash_attn_bhsd_auto_pipeline_h16_d128.py	√	√	√	×	√	×	√	60%

算子实现：https://github.com/tile-ai/tilelang-ascend/tree/ascendc_pto/examples/flash_attention/fa_opt

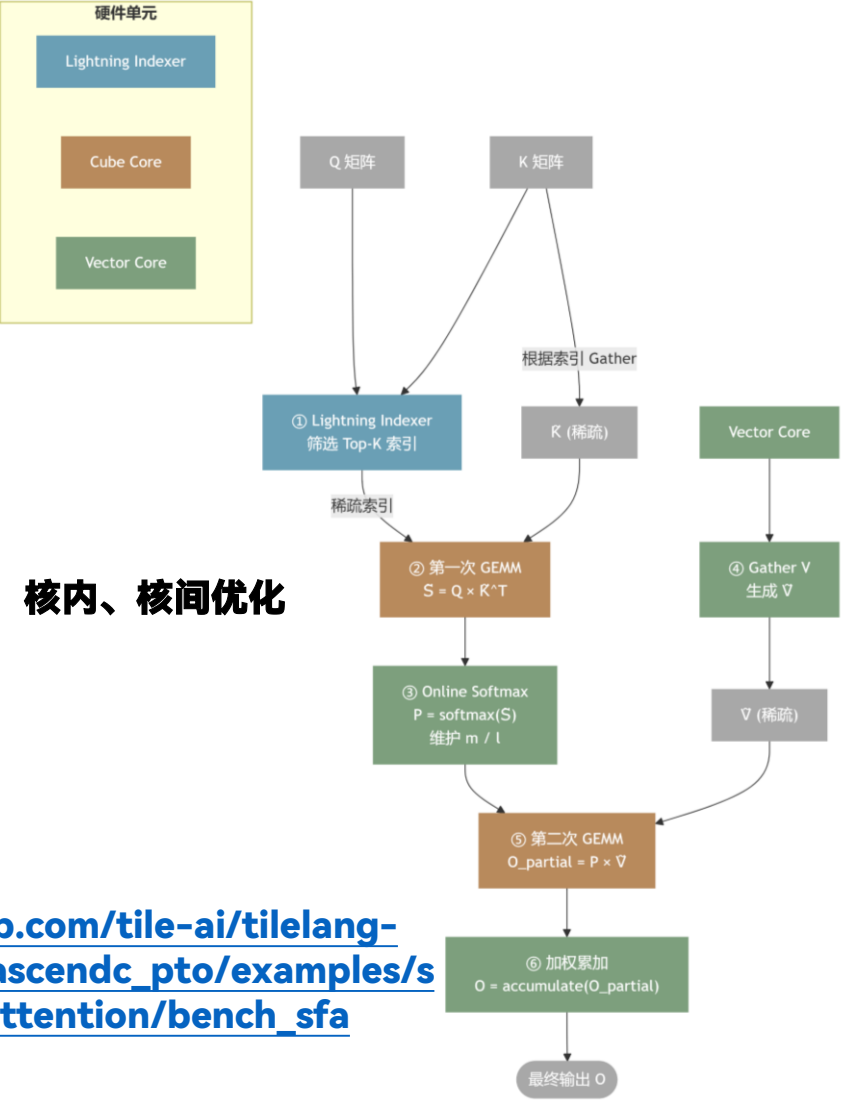
文件名	说明
flash_attn_bhsd_expert_h16_d128.py	Expert 模式最佳性能实现 (80%)
flash_attn_bhsd_auto_pipeline_h16_d128.py	混合编程模式实现 (60%)
flash_attn_bhsd_ascendc.py	AscendC 原生算子调用脚本
bench_test.sh	性能精度测试脚本

SFA算子性能优化案例

Sparse FlashAttention (SFA) 将 FlashAttention 的 IO 感知融合计算与稀疏注意力相结合，旨在打破长上下文处理的瓶颈。它通过仅计算注意力矩阵中的“关键”部分，将计算复杂度从 $O(L^2)$ 降低至约 $O(L \log L)$ ，同时保持了 FlashAttention 在内存访问上的高效性。

Name	Last commit message
..	
figures	[docs] enhance sfa optimization documentation
README.md	[docs] enhance sfa optimization documentation
README_zh.md	[docs] enhance sfa optimization documentation
bench_sfa.py	[Perf] Improve SFA Performance and Add Optimization Guide (#665)
sparse_flash_attn_pa.py	[Feature] Remove tmp argument in vector core API (#676)
sparse_flash_attn_pa_baseline.py	[Feature] Remove tmp argument in vector core API (#676)
sparse_flash_attn_pa_developer.py	[Feature] Remove tmp argument in vector core API (#676)
sparse_flash_attn_pa_no_cv_pipeline.py	[Feature] Remove tmp argument in vector core API (#676)
sparse_mla_performance_optimization.md	[docs] enhance sfa optimization documentation
sparse_mla_performance_optimization_zh.md	[docs] enhance sfa optimization documentation

<https://gitcode.com/cann>



CV融合算子：核内、核间优化

https://github.com/tile-ai/tilelang-ascend/tree/ascendc_pto/examples/parse_flash_attention/bench_sfa



SFA算子性能优化案例

典型shape下开箱性能为Ascend C手写算子的13%：

shape	TileLang(us)	Ascend C(us)	性能现状
t: 1, b: 1, q_n: 128, kv_n: 1, d: 512, d_rope: 64, sparse_size: 128, block_size: 128, act_kv_size: 2560	737	98	13.3%
t: 1, b: 1, q_n: 128, kv_n: 1, d: 512, d_rope: 64, sparse_size: 128, block_size: 128, act_kv_size: 6400	735	99	13.47%
t: 1, b: 1, q_n: 128, kv_n: 1, d: 512, d_rope: 64, sparse_size: 128, block_size: 128, act_kv_size: 48000	736	100	13.57%

优化后典型shape下性能达Ascend C手写算子的90%：

shape	TileLang(us)	Ascend C(us)	性能现状
t: 1, b: 1, q_n: 128, kv_n: 1, d: 512, d_rope: 64, sparse_size: 128, block_size: 128, act_kv_size: 2560	109.760	98	89.2%
t: 1, b: 1, q_n: 128, kv_n: 1, d: 512, d_rope: 64, sparse_size: 128, block_size: 128, act_kv_size: 6400	109.100	99	90.7%
t: 1, b: 1, q_n: 128, kv_n: 1, d: 512, d_rope: 64, sparse_size: 128, block_size: 128, act_kv_size: 48000	108.980	100	92.6%

SFA算子性能优化案例

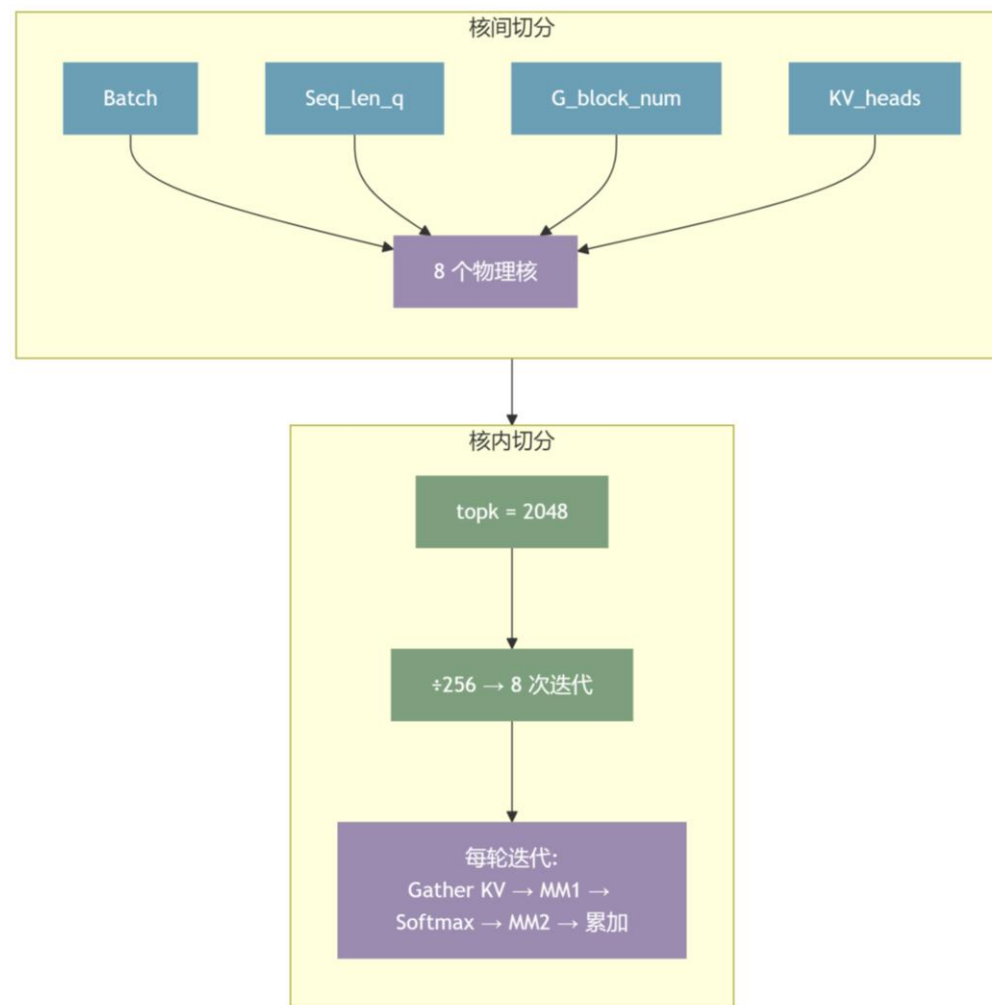
➤ 数据切分 + Fixed Core

• 核间切分

SparseMLA 的核间切分沿 $\text{batch} \times \text{seq_len_q} \times \text{g_block_num} \times \text{kv_heads}$ 四个维度展开，将任务固定在8个物理核上。

• 核内切分

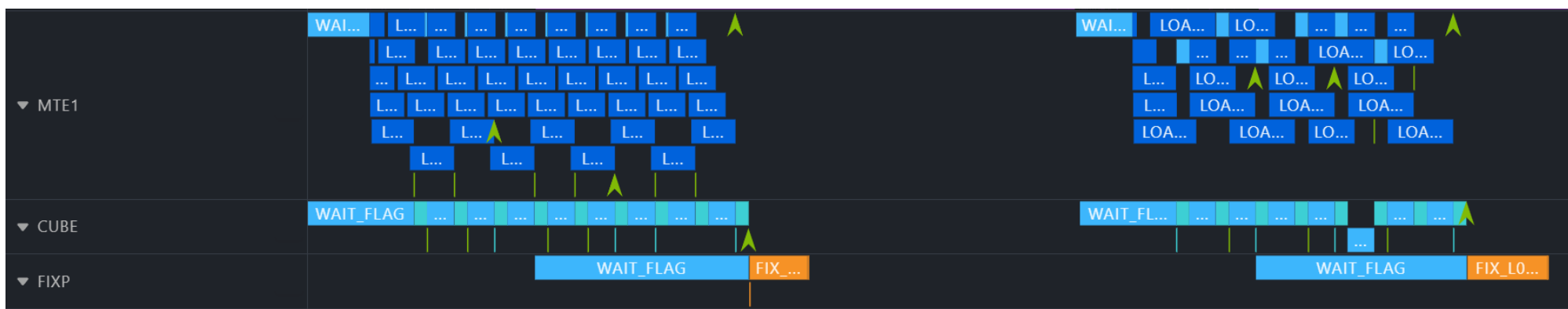
每个物理核在执行单个逻辑任务时，沿 $\text{seq_len_kv}(\text{topk})$ 维度进一步分块迭代，将 $\text{topk}=2048$ 按 $\text{n_base_size}=256$ 切分，得到 $\text{n_block_num} = \text{topk} / \text{n_base_size} = 8$ 次迭代。每次迭代执行一轮完整的 $\text{Gather KV} \rightarrow \text{MM1}(\text{QK}^T) \rightarrow \text{Softmax} \rightarrow \text{MM2}(\text{PV}) \rightarrow$ 累加输出流程。



SFA算子性能优化案例

➤ Cube核内切分: SplitK+Pipeline

- **MM1 (Q×K^T) 的 K 维切分:** 矩阵乘的 K 维度为 $\text{dim} + \text{rope_dim} = 512 + 64 = 576$, 按 $\text{k_l0_size}=64$ 切分为 $576/64 = 9$ 个子块。L0A/L0B 各申请双份缓冲($\text{q_l0a}[2, 16, 64]$ 、 $\text{kv_l0b}[2, 64, 256]$) , 通过 Ping-Pong 实现 MTE1 搬运与 Cube 计算的流水重叠。
- **MM2 (P×V) 的 K 维切分:** 矩阵乘的 K 维度为 **$\text{n_base_size} = 64 \rightarrow 256$** , 尽量占满L1, 按 $\text{mm2_k_l0_size}=32$ 切分为 8 个子块分多次搬入L0, 同样采用双缓冲进行流水化。



双缓冲通过 $\text{kk} \% 2$ 交替写入 L0 缓冲区, 并以 $\text{set_flag} / \text{wait_flag}$ 进行精确同步: 当 Cube 正在对第 i 份 L0 数据执行 mma 运算时, MTE1 同时将第 $i+1$ 份数据从 L1 搬入另一半缓冲, 实现流水重叠。

SFA算子性能优化案例

➤ Vector完成稀疏访存优化

在稀疏场景下，KV 数据在 Global Memory 中呈离散分布，但后续的 Cube 矩阵运算需要连续的内存输入，使用Vector核做前处理，将非连续的数据搬运连续后再通过workspace交互至Cube核运算。

优化手段	核心说明
双 Vector 核指令发射	利用 A2/A3 芯片 Cube:Vector = 1:2 的特性，由 Vector 核发出访存指令，指令下发效率提升一倍。
离散 Gather + 连续搬出	统一稀疏索引将离散 KV 从 GM 收集到 UB，连续化处理后一次性搬入 Workspace，供 Cube 核直接使用。
异步拷贝（核内手动同步）	使用 set_flag / wait_flag 精确控制异步拷贝，离散拷入时间大幅缩减。
Ping-Pong 双缓冲	当前块 MTE3 写出的同时，下一块的 MTE2 读取已异步发起，搬入搬出流水重叠。

```
for bi_i in range(BI // 2):
    index_i = indices_ub[bi_i + vid * BI // 2]

    block_idx = index_i // block_size
    block_i = block_table[b_i, block_idx]
    block_inter = index_i % block_size

    T.copy(KV[block_i, block_inter, 0, :D], kv_ub_gather[bi_i, :])
    T.copy(KV[block_i, block_inter, 0, D:], kv_tail_ub_gather[bi_i, :])

T.copy(kv_ub_gather, workspace_1[cid, vid * BI // 2 : (vid + 1) * BI // 2, :])
T.copy(kv_tail_ub_gather, workspace_2[cid, vid * BI // 2 : (vid + 1) * BI // 2, :])

for i_i in T.serial(NI):
    T.copy(workspace_1[cid, 0:BI, 0:D], kv_l1)
    T.copy(workspace_2[cid, 0:BI, 0:D_tail], kv_tail_l1)

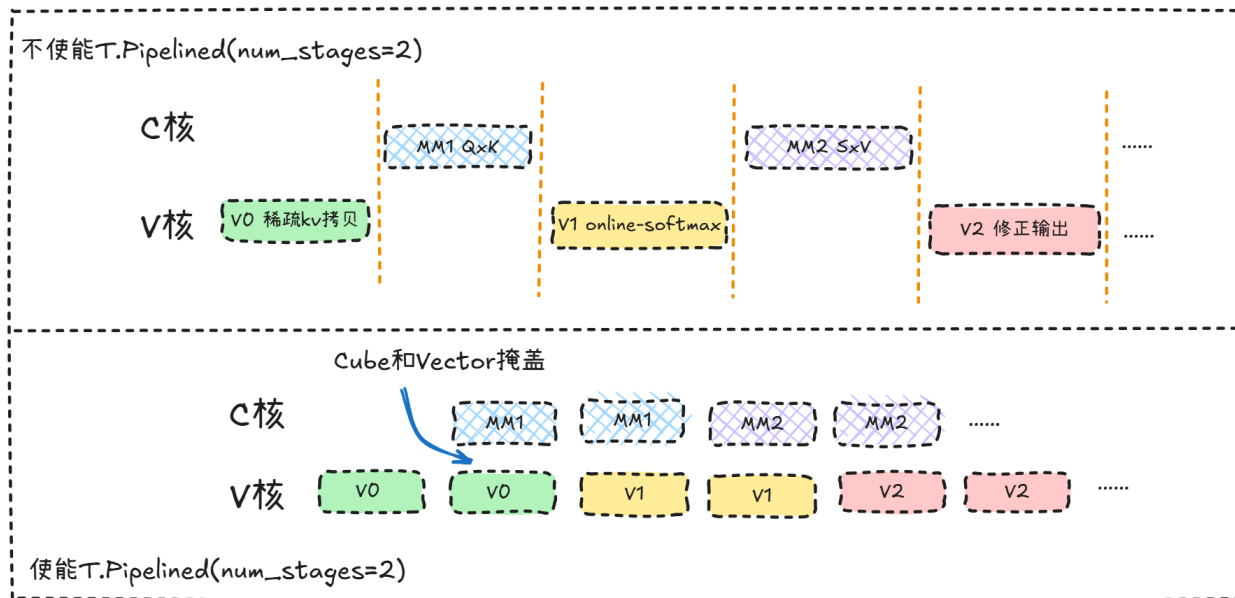
    T.gemm_v0(q_l1, kv_l1, acc_s_l0c, transpose_B=True, init=True)
    T.gemm_v0(q_tail_l1, kv_tail_l1, acc_s_l0c, transpose_B=True)

T.copy(acc_s_l0c, workspace_3[cid, 0:H_per_block, 0:BI])
T.copy(workspace_4[cid, 0:H_per_block, 0:BI], acc_s_l1)
```


SFA算子性能优化案例

核间Pipeline优化

下一次迭代的 Vector MTE 访存(如 V0 阶段的 kv gather 与写出) 被与上一次迭代的Cube核计算(C1阶段 QK 矩阵乘) 掩盖、下一次迭代的vector计算(V1 阶段 Online Softmax) 与上一次迭代的Cube核计算(C2阶段 PV 矩阵乘) 掩盖等。通过不同硬件单元相互掩盖，大幅度减少算子耗时。



num_stages=2, cross_interval = 1

<https://gitcode.com/cann>



SFA算子性能优化案例

➤ 优化Vector算法，减少指令数: Axpy

```
# 优化前: 两条指令
T.tile.mul(acc_s_ub, acc_s_ub, sm_scale)
T.tile.sub(acc_s_ub, acc_s_ub, m_i_2d)

# 优化后: 一条指令
T.tile.axpy(acc_s_ub, m_i_2d, sm_scale)
```

➤ 离散访存连续化: broadcast+vec

```
# 优化前, 对m_i进行多次getvalue的scalar操作
for h_i in range(block_M // 2):
    T.tile.sub(acc_s_ub[h_i, :], acc_s_ub[h_i, :], m_i[h_i])

# 优化后, 转化为一次tile操作
T.tile.broadcast(m_i_2d, m_i, tmp_ub)
T.tile.sub(acc_s_ub, acc_s_ub, m_i_2d)
```

act_kv_size = 2560:

文件	Fixed Core	s2切分大小	kv gather与连续搬出	异步拷贝/核内手动同步	搬入搬出 Ping-Pong	CV pipeline	Broadcast 优化	AXPY 优化	性能数据 (A3)
sparse_flash_attn_pa_baseline.py	√	64	×	×	×	×	×	×	602us
sparse_flash_attn_pa_developer.py	√	64	√	×	×	×	√	×	347us
sparse_flash_attn_pa.py	√	64	√	√	√	√	√	√	127us
sparse_flash_attn_pa_no_cv_pipeline.py	√	256	√	√	√	×	√	√	109us

Baseline: 13%

16.4%
28.2%
77.2%
90.8%



总结

FA算子:

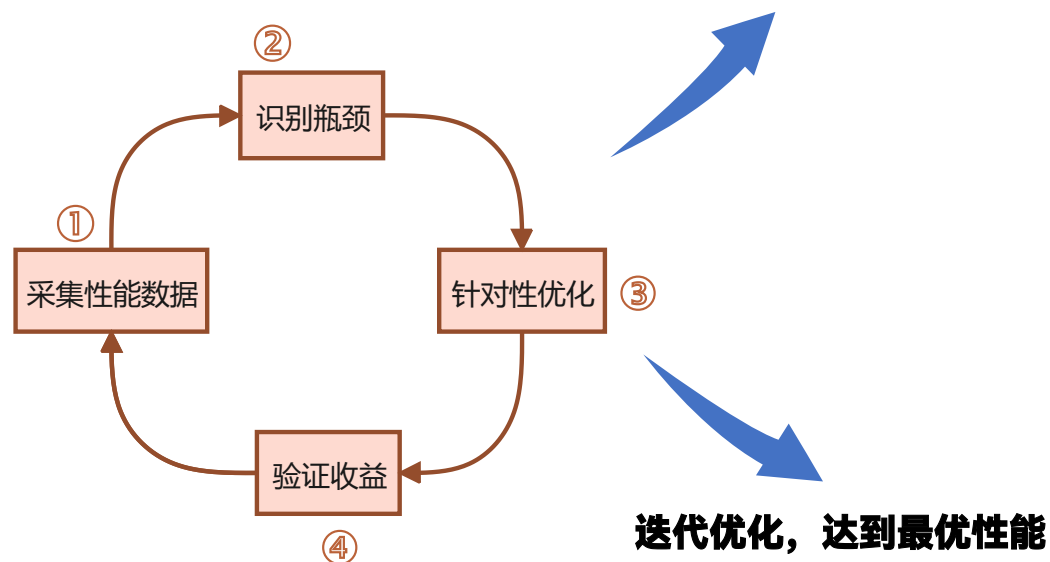
- **Cube:** gemm同步细化
- **vector:** double buffer、算法优化、指令向量化
- **核间:** num_stages=8, cross_interval=2

SFA算子:

- **Cube:** splitK、调整切分策略使L1满载
- **Vector:** double buffer、算法优化、指令向量化、离散访存连续化
- **核间:** num_stages=2, cross_interval=1

<https://gitcode.com/cann>

原则	说明
掩盖短流水	将耗时短的流水尽量用耗时长流水掩盖
减少气泡	优化任务排布, 减少任务间等待时间
单一 Bound	理想情况下优化至单一类型流水 bound



CANN

Thank you.

社区愿景：打造开放易用、技术领先的AI算力新生态

社区使命：使能开发者基于CANN社区自主研究创新，构筑根深叶茂、跨产业协同共享共赢的CANN生态

Vision: Building an Open, Easy-to-Use, and Technology-leading AI Computing Ecosystem

Mission: Enable developers to independently research and innovate based on the CANN community and build a win-win CANN ecosystem with deep roots and cross-industry collaboration and sharing.



上CANN社区获取干货



关注CANN公众号获取资讯